8. Extend a Binary Search Tree program to support deletion. The program should create a BST, delete a user-specified node, correctly handle nodes with zero, one and two children, and display inorder traversal before and after deletion. Test the program separately for all three deletion cases.

Source code :-

```
#include <stdio.h>

#include <stdlib.h>

struct Node

{

   int data;

   struct Node *left;

   struct Node *right;

};

struct Node* createNode(int value)

{

   struct Node *newNode;

   newNode = (struct Node*)malloc(sizeof(struct Node));

   newNode->data = value;

   newNode->left = NULL;

   newNode->right = NULL;

   return newNode;

}
```

```
struct Node* insert(struct Node *root, int value)
{
   if (root == NULL)
      return createNode(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);
   return root;
}
void inorder(struct Node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}
struct Node* findMin(struct Node *root)
{
   while (root->left != NULL)
      root = root->left;
   return root;
}
struct Node* deleteNode(struct Node *root, int key)
{
   struct Node *temp;
   if (root == NULL)
      return root;
```

```
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else
{       /* Case 1: No children */
        if (root->left == NULL && root->right == NULL)
        {
            free(root);
            return NULL;
        }
        /* Case 2: One child */
        if (root->left == NULL)
        {
            temp = root->right;
            free(root);
            return temp;
        }
        if (root->right == NULL)
        {
            temp = root->left;
            free(root);
            return temp;
        }
        /* Case 3: Two children */
        temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
```

```
    return root;
}

int main()
{
    struct Node *root = NULL;
    int n, value, key, i;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &value);
        root = insert(root, value);
    }
    printf("\nInorder before deletion: ");
    inorder(root);
    printf("\nEnter node to delete: ");
    scanf("%d", &key);
    root = deleteNode(root, key);
    printf("\nInorder after deletion: ");
    inorder(root);
    printf("\n");
    return 0;
}
```

Output :-

